DepWeaver：以多來源融合、信心分級與對話式缺口補齊建構微服務相依圖

DepWeaver: Multi-Source Fusion with Confidence Grading and Conversational Gap-Filling for Microservice Dependency Graph Construction

***摘要*—微服務系統的相依圖是部署排序、變更影響分析與測試規劃的基礎，但單一來源難以完整重建它：執行期觀測只看得到被觸發的呼叫，靜態分析處理不了動態分派，也處理不了尚未支援文法的語言，專案文件則可能過時或錯誤。把多個來源合在一起可以提高完整度，卻帶來新的問題：讀者分不出圖上哪一條關係是量測到的、哪一條只是有人提過，覆蓋率數字與 LLM 撰寫的報告也可能和圖互相矛盾。本文提出 DepWeaver，一個整合在 ChatOps 環境中的相依圖建構方法。它從服務網格的執行期觀測、tree-sitter 靜態分析與專案文件三個來源抽取關係，再融合成統一模型；模型中每條邊帶有逐條出處與三級信心等級（僅推測 < 有使用證據 < 量測到）。圖、覆蓋率與報告都由這個模型產生：相依關係清單由程式撰寫，不交給 LLM 重述；覆蓋率只計算量測機制能夠量測的關係；沒有量測的值明示為「未量測」。自動手段用盡時，才在對話中請操作者提供缺少的值，並確認對不上的服務名稱指的是哪個服務。在 19 個開源系統與 1 個私有系統上評估：靜態層對第三方 MicroDepGraph 參考邊集合的吻合率為 0.98（93/95）；作為假想敵，不給檔案的 LLM 直接畫圖 F1 為 0.70，給了檔案後為 0.90，但沒有出處、執行結果不穩定、名稱常對不上，而它贏過靜態層的地方正好指出規則的缺口。補上規則後，歸納用的 7 個系統 precision 全為 1.00、recall 0.99，但這些規則對 3 個未見過的 hold-out 系統一條邊都沒有改變；改從 hold-out 歸納、再以另外 2 個未見過的系統量測，才有部分轉移（其中一個系統的業務呼叫邊 recall 由 0.11 提高到 0.89）。靜態層的 recall 因此是寫法涵蓋率，不是靜態分析的極限。在真實部署的 Bank of Anthos 上，融合執行期觀測後業務呼叫覆蓋率 7/7、資料層 5/5；關閉 repo 範例來源再執行一次時，最後兩條業務邊由對話補齊；操作者回答 3 個名稱問題後，一個系統的吻合率由 24/26 提高到 26/26。**

***Index Terms*—微服務、架構重建、相依圖、靜態與動態分析、服務網格、出處追溯（provenance）、ChatOps、大型語言模型、human-in-the-loop**

I. 緒論

A. 動機

微服務系統常由數十到數百個獨立部署的服務組成，服務之間透過 HTTP、gRPC、訊息佇列與共用資料庫彼此相依。這張相依圖決定了部署順序、變更的影響範圍，以及需要補強的測試，但它通常沒有完整的紀錄：架構圖在發布後容易逐漸過時，程式碼散落在不同的 repo 與語言裡，真正執行的流量又只覆蓋其中一部分。

自動重建相依圖的方法可依來源分成三類。動態方法從分散式追蹤、服務網格或網路流量看到的都是真的發生過的呼叫，但只看得到被觸發的路徑 [1], [2]。靜態方法讀取原始碼、設定檔與部署描述檔，涵蓋範圍較廣，但受限於語言文法、動態分派與「宣告了卻沒用到」的設定 [3], [4], [5]。文件或 LLM 方法能補充前兩者看不到的關係，但可靠度較低；例如讓 LLM agent 自行讀程式碼重建微服務架構，受測的兩個模型中較好的 GPT-4o 平均 F1 約 0.6 [6]。已有研究量化了單一來源的不足：靜態呼叫圖相對於實際執行的 recall 中位數約 0.88 [7]；一份研究比較了 9 個靜態架構重建（architecture recovery）工具，表現最好的單一工具 F1 為 0.86，依特徵選出最佳組合（共四個工具）後 F1 達 0.91 [8]。

這些結果顯示，有必要融合多個來源。但融合之後出現三個新問題，這是本文處理的問題：

1) **等級被抹平。**文件只提過一句的關係，和執行期量測到數千次的關係，在圖上的呈現方式相同，讀者無從判斷哪條可以據以決策。
2) **產出物互相矛盾。**圖、覆蓋率與 LLM 撰寫的報告各自從原始來源推導，報告把資料庫邊寫成「執行期是否觀測：未知」，圖卻畫實線；覆蓋率的分母混進無法量測的項目，使分數虛低或虛高。
3) **有些缺口只有人知道答案。**要觸發某條路徑，需要只有操作者知道的值（例如這個部署的密碼）；文件中的某個名稱指的是哪個服務，規則與 LLM 都只能推測。

B. 方法概要與貢獻

DepWeaver 把相依圖的建構拆成「收集、融合、產出、補齊」四個階段，並遵守一條原則：能由程式從證據算出來的，就不交給 LLM；LLM 只處理規則未涵蓋的部分與報告的敘述。它以 ChatOps 的形式運作：操作者在 Discord 輸入一個斜線指令，工具在背景收集證據、產生流量、查詢量測指標，遇到只有人知道的事就在同一個對話中提問，最後貼回一張分層相依圖、覆蓋率與報告。本文的貢獻如下：

1) **三個來源、三級信心等級的融合模型**（第 III-B、III-C 節）：執行期觀測、靜態分析與文件以（來源，目標）為鍵合併；每條邊帶一個有序的信心等級與完整的出處（provenance）清單，合併時等級取最高、出處取聯集。
2) **由同一模型產生、彼此一致的產出物**（第 III-E 節）：列出相依關係的事實段落由程式產生後插入 LLM 撰寫的報告；覆蓋率的分母只計算量測機制能夠量測、且信心等級高於「僅推測」的關係，僅推測的邊的數量寫在分數旁邊；未量測的狀態以三值表示。
3) **對話式缺口補齊：自動手段用盡後才詢問操作者**（第 III-F 節）：流量參數依序從 repo 範例、錯誤回饋、詢問操作者取得；對不上的名稱不丟棄、不模糊比對，而是附排序過的候選請操作者確認，答案以 repo 為單位保存。

4) **在 19 個開源系統與 1 個私有系統上的評估**(第 IV 節):包含一個純 LLM 直接畫圖的假想敵對照、一份非作者標註的第三方參考邊集合、逐條可複驗的作者參考邊集合、三輪「在一組系統上歸納規則、在下一組未見過的系統上量測轉移」的 hold-out 評估、一個真實部署的執行期驗證，以及詢問操作者前後的對照。

C. 研究問題

- **RQ0（假想敵）**：不用 DepWeaver，直接讓 LLM 畫相依圖，在不給事實與給事實兩種條件下各能畫得多正確？它的輸出缺什麼？
- **RQ1（靜態層的泛化）**：只用靜態來源，DepWeaver 在沒有調校過的系統上能重建多少相依關係？錯誤來自哪些原因？
- **RQ2（融合執行期觀測）**：加入執行期觀測後，信心等級與覆蓋率如何變化？
- **RQ3（對話式缺口補齊）**：自動手段用盡之後詢問操作者，能補回多少相依關係？需要詢問多少問題？
- **RQ4（產出物的一致性）**：第 III-C、III-E 節的設計規則防止了哪些實際發生過的錯誤？

II. 背景與相關研究

A. 微服務架構重建

靜態方法方面，Bushong 等人以靜態分析重建整個微服務系統的架構 [3]；Code2DFD 在原始碼與設定檔中尋找關鍵字作為證據，產生資料流圖 [4]；MicroDepGraph 從 docker-compose 中定義的相依關係與 Java 原始碼中的內部 API 呼叫取得相依關係 [5]。動態方法從系統已在收集的日誌、追蹤與指標重建執行期視圖 [1]，相關的視覺化研究見 [9]；也有方法直接觀察 socket，取得行程之間的網路相依關係 [2]。混合方法結合靜態來源與執行期資訊 [10], [11]，例如 Soldani 等人以 Kubernetes manifest 取得節點、以 Kiali 先前產生的執行期圖取得邊 [12]；Mayer 與 Weinreich 的理由是有些關係只在執行期才看得到 [10]。

既有的比較研究與文獻回顧顯示：37 個重建工具中 19 個純靜態、10 個純動態、只有 8 個結合兩者 [13]；依特徵挑選四個靜態工具組合起來，比表現最好的單一工具好 [8]；2026 年的文獻回顧把「可靠地融合異質證據」列為未解的問題 [14]。在微服務之外，SARIF 依各來源的相關性與品質，融合相依關係、程式碼文字與目錄結構 [15]。本文的差別在於：上述融合方法多半只輸出邊的聯集或分群結果；DepWeaver 保留每條邊由哪個來源、以什麼信心等級支撐，並讓下游產出物依等級決定是否計分。

B. 服務相依圖的分析與測試

Ma 等人提出以服務相依圖分析與測試微服務的 GMAT [16]，之後加入服務與情境之間的邊以支援回歸測試的案例檢索 [17]，並以版本為單位建構相依圖，在設計期標出設計問題、在執行期偵測異常 [18]；MicroViSim 針對 Kubernetes 上的微服務系統，從使用者定義的 YAML 設定產生服務相依圖與負載模擬 [19]。這些工作以相依圖為輸入；本文處理的是如何從多個來源完整建構相依圖本身，並讓讀者知道每條邊有多可信。

C. LLM 與架構重建

Quadri 等人結合從程式碼建構的知識圖譜與 LLM 推理做架構重建 [20]。另一方面，讓 LLM agent 自行重建微服務架構時，受測的兩個模型中較好的 GPT-4o 平均 F1 約 0.6 [6]；第 IV-B 節在本文的參考邊集合上重做這個對照，並區分「不給檔案」與「給檔案」兩種條件。在生成端，Satyrn 先以分析程式算出事實再交給 LLM 撰寫，報告中正確陳述的比例超過 86%，直接讓 GPT-4 Code Interpreter 撰寫只有 57% [21]；PAL 讓 LLM 寫程式、由直譯器計算答案 [22]。DepWeaver 採取同樣的分工。

D. Human-in-the-Loop 與 ChatOps

Reflexion model 讓工程師定義高階模型與它到原始碼的對應，工具據此算出模型與程式碼一致和不一致之處 [23], [24]；文獻回顧也指出，混合式的重建工具鏈必須允許人工驗證與修正不完整的結果 [14]。部分業界工具讓使用者事後撰寫改名、合併、拆分服務的規則，例如 Datadog 的 service remapping [25]；DepWeaver 則在分析進行中、在同一個對話裡提出具體問題。ChatOps 方面，MSABot [26] 與 MsdoBot [27] 以聊天機器人支援微服務的開發與維運；ChatOps4Msa 以低程式碼的服務組合建構微服務 ChatOps [28]，後續版本加入 LLM [29]；PipelineBot 以 LLM 驅動的 ChatOps 自動化部署管線與監控 [30]。DepWeaver 建在 ChatOps4Msa 之上，處理的是不同的問題：相依圖的建構，以及在建構中向操作者取得只有人知道的資訊。

E. 呈現不確定性

Guo 等人以受控實驗比較在圖的邊上同時呈現主屬性與不確定性的視覺變數 [31]；partial models 主張模型應能表達建模者的不確定性 [32]。部分業界服務圖會區分直接量測與推測的節點，例如 AWS X-Ray 以下游呼叫產生 inferred segment，等該服務自己送出資料時再取代它 [33]。DepWeaver 把這個二分擴充成三級，並把等級帶進覆蓋率與報告。

III. 方法

A. 整體流程

圖 1 是 DepWeaver 的流程。操作者以斜線指令給定 repo、namespace 與入口 URL；工具在背景執行收集管線，每個階段的結果寫入 JSON 檢查點。收集結束後，工具依確定性計算的覆蓋率，決定是否針對缺口再送一輪流量；需要只有人知道的資訊時暫停，並在對話中提問。收集期間會先合併一次，用來計算覆蓋率並找出要詢問的名稱；操作者按下 Generate report 後，再從檢查點建立統一模型（canonical model）並產生所有產出物。

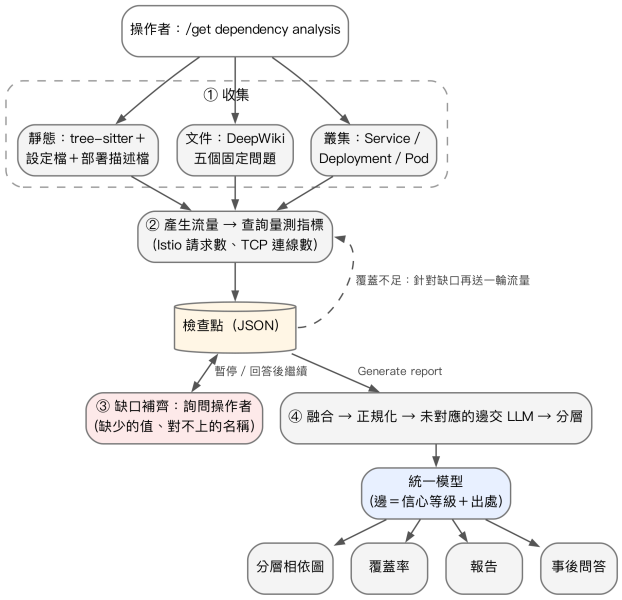


圖 1. DepWeaver 的流程。收集與產出以檢查點分開，缺口補齊與補送流量都可以多輪進行

兩個運作模式共用同一條流程。**有叢集**時，收集三個來源以及叢集的部署狀態；**沒有叢集**（namespace 為 none）時，需要叢集的步驟被跳過，只產出由靜態來源與文件構成的草圖（以下稱無叢集模式），並在每個受影響的產出物上註明「未量測」。

B. 三個來源

執行期觀測。系統執行時，Istio 為每個服務部署一個 proxy（sidecar），服務之間的每一次呼叫都經過它；proxy 記下「誰呼叫了誰、幾次、成功與否」並寫入 Prometheus。DepWeaver 查詢三組計數：HTTP 與 gRPC 請求（istio_requests_total，以接收端回報為準）、服務網格內的 TCP 連線（istio_tcp_connections_opened_total；資料庫這類非 HTTP 的連線只在 TCP 層可見），以及目的地在網格外的 HTTP 與 TCP 流量（外部相依關係）。

靜態分析。以 tree-sitter 解析 Java 與 Python 原始碼，用查詢檔比對 HTTP client、Feign、RestTemplate、訊息佇列，以及持久化 annotation 與模型宣告（JPA 的 @Entity 與 Repository、SQLAlchemy、Django ORM）；其他語言則改由 LLM 閱讀程式片段。設定檔（Spring 的 application.yml、Spring Cloud Config 的設定 repo）、部署描述檔（Kubernetes manifest、Docker Compose）與反向代理設定（nginx、Apache）由確定性解析器讀取位址與連線設定。每條抽取到的關係附「檔案：行號」。

專案文件。透過 DeepWiki 對 repo 提出五個固定問題（服務職責、同步呼叫、非同步呼叫、外部相依關係、架構流程（入口、請求流程與相依樞紐）），再由 LLM 整理成結構化清單。有些呼叫只在罕見流程中發生，只有文件會提到；但文件也是最可能過時或錯誤的來源。

C. 信心等級、出處與合併

信心等級。每條邊帶一個有序的三級信心等級（表 I），Graphviz 圖上以點線、虛線、實線呈現（圖 2）；Mermaid 沒有點線，僅推測的邊改以虛線加「?」標示。注意「有使用證據」（documented）指有程式碼或設定佐證，不是「文件提到」。等級與出處分開儲存：一條邊可以有多筆出處（程式碼的某一行、某個指標、文件的某一段），但只有一個等級。

表 I　三級信心等級（由低到高）

等級	條件	範例
僅推測（inferred）	只有文件敘述而無設定佐證、只有資料庫連線設定而無持久化程式碼、workload 本身環境變數中的服務位址，或由變數名推得	有 datasource URL 但沒有持久化程式碼
有使用證據（documented）	有會發出呼叫的程式碼，或設定檔中指向該服務的位址（workload 本身環境變數除外）；Compose 的 depends_on／links；文件中有設定佐證的關係；資料庫連線設定加上持久化程式碼	Feign client 位於 CustomersServiceClient.java:36
量測到（observed）	執行期觀測記錄到這條關係發生過	istio_requests_total 記錄 ingress 到 api-gateway 共 121 次（petclinic，2026-07-29 執行期邊清單）

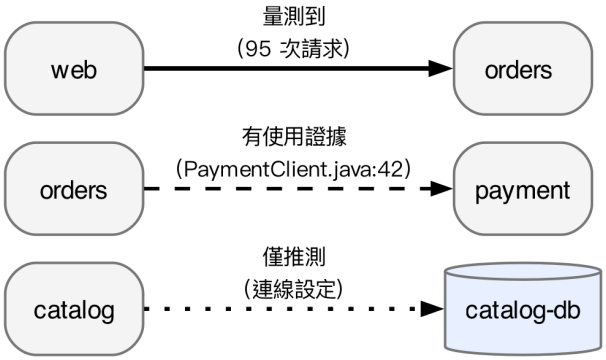


圖 2. 三級信心等級在 Graphviz 圖上的呈現（示意）：點線（僅推測）、虛線（有使用證據）、實線（量測到）

合併規則如演算法 1 所示：以（來源，目標）為鍵，等級取最高、出處取聯集。名稱對齊不在這一步進行：各合併器在呼叫 addEdge 之前先對齊名稱；正規化改名時，再透過同一個函式把邊合併回去。下游產出物只能沿用或降低等級，不能提高。我們選擇離散的三級而不是 0 到 1 的連續值，原因是不需要校準參數或加權公式，且可直接對應三種線型。代價是無法表達「兩個弱來源互相佐證」。

演算法 1：邊的合併（addEdge）

```
輸入：模型 G，候選邊 e = (s, t, level, layers, refs, observed, count)；s、t 已由
    合併器對齊名稱
1: if (s, t) ∉ G.edges then
2:     G.edges[(s, t)] ← e; return
3: end if
4: old ← G.edges[(s, t)]
5: old.level ← max(old.level, e.level)       // inferred < documented < observed
6: old.layers ← old.layers ∪ e.layers        // runtime／code／doc
7: old.refs ← old.refs ∪ e.refs              // 出處取聯集
8: old.observed ← old.observed ∨ e.observed
9: old.count ← max(old.count, e.count)       // 只有量測來源帶次數
```

資料庫邊的升級條件。資料庫邊只有連線設定時是「僅推測」；同一個服務同時找到持久化程式碼，才升為「有使用證據」；在服務網格內量測到 TCP 連線後，才升為「量測到」。Spring Boot 以環境變數 SPRING_DATASOURCE_URL 隱式綁定資料來源，程式碼裡沒有讀取這個值的行，因此 DepWeaver 也讀取 Deployment 的 envFrom 與 configMapKeyRef，把注入的位址當成連線設定。

合併順序。先以執行期觀測建立骨架；再依序疊加程式碼邊（先對齊節點，對不上的列入未對應清單）、文件邊（不新增服務節點，對不上的服務名稱改為詢問；但可新增資料庫、佇列與外部主機節點）、持久化證據，以及叢集的部署狀態。

D. 正規化、未對應的邊與分層

合併後的圖帶有三類雜訊：框架函式庫被當成服務、同一服務的多種拼法（模組目錄名、部署名、文件中的名稱），以及控制平面元件。DepWeaver 以 46 條不含任何專案名稱的抽取與合併規則，加上框架函式庫與控制平面的名單處理；表 II 列出其中幾條代表性的規則；每條規則都是在第 IV-B 節的某個系統上逐條比對差異後歸納出來的，並有對應的單元測試（46 條全表見附錄；它們分四輪歸納，每一輪對未見過的系統轉移了多少，見第 IV-C 節）。

表 II　抽取與合併規則（節錄）

規則	歸納出此規則的系統
無叢集時，以 Kubernetes workload 名稱作為服務名稱清單	ewolff、TeaStore、Online Boutique
模組目錄名與 workload 名稱完全相同，或以「-部署名」結尾時，視為同一服務	ewolff、TeaStore
佔位符依序查環境變數、屬性、預設值，最後才用變數名（標為僅推測）	piggymetrics、ewolff、robot-shop
單獨的主機名變數（如 DB_HOST）也視為呼叫目標	piggymetrics、TeaStore、robot-shop
反向代理設定視為入口的邊清單	ewolff、robot-shop
Compose 描述的是服務之間的連線設定，不是某個服務的設定；depends_on 的目標也建立節點	TeaStore、robot-shop、ecommerce
localhost 永遠不是節點；未被引用的常數不算呼叫	ecommerce
workload 本身環境變數中的服務位址畫成僅推測	Online Boutique、Bank of Anthos、TeaStore

未對應的邊交給 LLM。只有規則未涵蓋的少數程式碼邊會交給 LLM 判斷目標節點。LLM 回傳的來源必須是圖上已存在的服務；目標必須是既有節點，或明確標為外部主機、訊息佇列的新節點；不符合時丟棄，並記成待詢問的名稱（第 III-F 節）。

分層。節點依序排成：入口層（gateway，或沒有被任何服務呼叫的服務）、依最長路徑深度排列的業務服務、資料層、外部主機、沒有任何邊的節點。使用最長路徑而非最短路徑，是因為一個既被入口直接呼叫、又在呼叫鏈末端被呼叫的服務應該排在鏈的下方，否則箭頭會往回指。以 Tarjan 演算法找出強連通元件，並把每個元件收縮成單一節點，同一個環的成員因此在同一層。在 train-ticket（51 條邊）上，分層後 51 條邊全部由上往下，沒有向上或同層的邊。

E. 讓產出物彼此一致

一個模型、多個視圖。每次產出先建立一個統一模型，再由它產生所有產出物：Mermaid 與 Graphviz 圖、覆蓋率、報告的事實段落、問答用的存檔。衍生計算（覆蓋率、分層）只實作一次。模型可序列化；報告發出數天後在 Discord thread 追問時，讀回的是同一份模型，不重新查詢叢集。

事實段落由程式產生。報告中列出相依關係的兩節（服務間的同步相依關係、基礎設施與資料層的相依關係）稱為事實段落，由程式從模型產生，措辭固定（例如資料庫邊寫「連線數」而非「請求數」，因為量測到的是前者），以 LLM 報告中第 6 節的標題為錨點插入；LLM 若自己也寫了這兩節，丟棄它的版本；找不到錨點時附在報告最後。「由操作者解析的名稱」一節也由程式產生，固定附在報告最後。其餘章節（收集狀態、元件、非同步通訊、未解項目與信心摘要）由 LLM 依各來源的原始紀錄與圖上的服務名稱撰寫。

只計可量測關係的覆蓋率分母。覆蓋率回答「應該被觸發的業務呼叫，有多少已被量測到」。演算法 2 列出分母的條件：量測機制能夠量測（兩端都已部署；業務呼叫不涉及平台的控制平面；資料層的目標不是行程內（in-process）元件），而且信心等級高於「僅推測」。業務呼叫（服務之間的 HTTP 與 gRPC）與資料層（目標為資料庫）分開計算；僅推測的邊的數量寫在分數旁邊；這個數量多於計分的邊時，加上「計分基礎過小」的警告；未覆蓋的邊列成清單，直接成為下一輪流量的目標。

演算法 2：覆蓋率分母

輸入：模型 G；輸出：計分集合 D_biz、D_db，僅推測邊數 M，未覆蓋集合 U

```
0: D_biz ← ∅；D_db ← ∅
1: M ← 所有 level = inferred 的邊的數量
2: for each e = (s, t) in G.edges do
3:     if e.level = inferred then continue
4:     if e 為服務間 HTTP／gRPC and deployed(s), deployed(t) and not
       controlPlane(s, t) then
5:         D_biz ← D_biz ∪ {e}
6:     else if kind(t) = db and deployed(s), deployed(t) and not inProcess(t) then
7:         D_db ← D_db ∪ {e}
8: end for
9: U ← {e ∈ D_biz ∪ D_db : ¬e.observed}
10: 報告 業務 |{e ∈ D_biz : e.observed}| / |D_biz|、資料層同理，附 M 與 U
11: if M > |D_biz| + |D_db| then 附加「計分基礎過小」警告
```

未量測不是零。部署狀態以可為 null 的布林值表示，預設「未知」，只有取得叢集資料後才設為否。沒有連上叢集時，報告的覆蓋率寫「not measured」，交給問答 LLM 的字串是 NOT MEASURED，兩處都不寫成 0%。抽取步驟失敗時記錄失敗與原因，不回傳空清單。

F. 對話式缺口補齊：最後才詢問操作者

流量參數。要讓執行期觀測看到一條呼叫，必須實際觸發它，而有些呼叫需要語意正確的輸入。DepWeaver 依序嘗試四層（表 III）。

表 III　流量參數的四層缺口補齊方式

層	做法	範例
1	從 repo 擷取範例請求（負載產生器、端對端測試、OpenAPI 與 Postman 範例）	Bank of Anthos 的 loadgenerator 提供存款欄位
2	把 4xx 回應交回 LLM 修正請求	缺欄位、格式錯誤
3	在 Discord 詢問操作者	部署的密碼、一個存在的收款帳號
4	操作者也無法提供：標記為無法觸發，不送出請求，在流量報告中列為缺口（仍留在覆蓋率分母中）	—

第 3 層的問題宣告在 Postman collection 中（空值變數加上說明）。沒有回答的值不綁定，需要它的請求暫不送出並標記為等待，不由 LLM 自行產生值。憑證必須整組詢問（只問帳號不問密碼時，產生器會自行編造密碼）；值不進入 prompt（LLM 只看到變數名）；密碼類欄位以遮罩顯示。答案存入檢查點後，自動針對缺口重送流量。流量產生器的 prompt 另外規定：操作回傳 200 但預期的相依邊沒有出現時，視為被拒絕，因為有些伺服器端渲染的應用拒絕表單時也回傳 200。

名稱對不上。需要詢問的名稱有四種來源：文件中的服務名稱經所有對齊規則仍無對應；程式碼殘餘邊的目標或所在模組目錄，經規則與 LLM 都無法對應（短名稱同時符合多個節點時，也是先交給 LLM）；名稱命中函式庫名單，卻有來自程式碼或部署層的邊；兩個服務名稱只差三字以上的前後綴，可能是同一服務。DepWeaver 把這些名稱各記成一個問題，不直接丟棄。候選依三個訊號排序（去除 service、api、client 等通用詞後的包含關係、共用字詞、編輯距離），排序只用於呈現，不做決定。操作者在表單中回答候選編號、任一拼法的服務識別字、new（新節點）或 ignore。答案以確定性規則解析，無法解析時保留問題，不加以猜測。若在表單中填入圖上不存在的節點，視為無法解析並保留問題；保存的舊答案若指向這次圖上不存在的節點，則不建立新節點、略過該名稱，避免系統改名後舊答案憑空產生節點。答案以 repo 為單位保存，不隨檢查點過期而清除，下一次分析同一個 repo 只詢問新出現的名稱；答錯可用斜線指令清除。報告多一節「由操作者解析的名稱」，由程式產生。

檢查點。收集、缺口補齊與產出之間以 JSON 檢查點分開：收集結果寫入後暫停，並貼出按鈕（Apply ServiceEntries、Resolve names、Provide values、Generate report、Pause & supplement；暫停後以 Resume 繼續）；操作者回答後，由新的一輪讀回檢查點繼續。一次分析因此可以跨越數小時與多次對話，而不需要長時間持有執行緒。

G. 實作

DepWeaver 是 ChatOps4Msa [29] 的一個 capability（Spring Boot 3.0、Java 17）：步驟宣告在 YAML 管線中，由既有的 orchestrator 依序呼叫；叢集操作透過 Kubernetes MCP server；LLM 使用 OpenAI 相容 API（實驗中為 gpt-4.1-mini，temperature 0）。測試套件共有 280 個測試案例（2026-10-20 執行）：270 個離線通過，9 個因需要外部環境或 API 金鑰而略過，1 個因需要連線至 MCP server 而失敗。原始碼、參考邊集合、15 份分析存檔（含 RQ2 五次執行期分析與 RQ3 五次實際執行的完整模型、覆蓋率訊息與報告）與重現指令將以匿名 artifact 提供。

IV. 評估

A. 受測系統

表 IV 列出受測系統。RQ0 與 RQ1 刻意挑選結構特性不同的新系統：非 Spring 的 Jakarta EE、以 enum 常數指定呼叫目標、再經自行實作的 registry 取得實例位址，五種語言混合、Helm 範本、以反向代理為入口。另有兩組在規則歸納前就選定、只用來量測規則轉移的 hold-out 系統（B 與 C），以及一個私有系統（AICS，13 個 repo 的 polyrepo，不可能在 LLM 的訓練資料裡）。

表 IV　受測系統

系統	語言	部署描述檔	用於
Bank of Anthos	Java、Python	Kubernetes	RQ0–RQ4
spring-petclinic	Java	Compose（RQ1）、Kubernetes（RQ4）	RQ1、RQ4
ewolff/microservice-kubernetes	Java	Kubernetes	RQ0、RQ1
ecommerce-microservice	Java	Compose	RQ0、RQ1
piggymetrics	Java	Compose	RQ0、RQ1
TeaStore	Java（Jakarta EE）	Kubernetes	RQ0、RQ1
Online Boutique	Go、C#、Node、Python、Java	Kubernetes	RQ0、RQ1
robot-shop	Node、PHP、Go、Python、Java	Helm、Compose	RQ0、RQ1
MicroDepGraph 中其餘 5 個系統	Java	Compose	RQ1；其中 spring-cloud-microservice 也用於 RQ3、RQ4
train-ticket	Java	Kubernetes	分層、回歸測試
opentelemetry-demo	Go、.NET、Node、Rust、Ruby、PHP、Kotlin、Java、Python、C++	Compose	RQ1 hold-out B
mall-swarm	Java（Spring Cloud Alibaba）	Compose、Kubernetes	RQ1 hold-out B
DeathStarBench hotelReservation	Go	Kubernetes、Compose、Helm	RQ1 hold-out B
yas	Java（Spring Boot 3）、TypeScript	Compose、Helm	RQ1 hold-out C
run-aspnetcore-microservices	C#（.NET 8）	Compose	RQ1 hold-out C
AICS（私有）	Python（FastAPI）	Kubernetes、Istio	RQ1（第 19 至 22 條規則的歸納來源）

模型與參數。所有涉及 LLM 的步驟都用同一個模型：OpenAI gpt-4.1-mini（Chat Completions API），temperature 0、n=1。DepWeaver 自身以它整理文件、閱讀無文法語言的程式片段與撰寫報告的敘述章節。RQ0 不用 system prompt，另加 JSON mode（response_format=json_object）與 max_tokens 12,000（第一批 9 次呼叫未設上限，其中 2 次達到模型預設上限 32,768，見 RQ0），每格 3 次執行，輸入預算為部署描述檔與設定檔 300 KB、原始碼 500 KB、單檔 20 KB，共 63 次呼叫、326 萬 prompt token。RQ2 的問答對照使用同一模型；檢索為 BM25 與 text-embedding-3-small 向量檢索以 RRF 融合、top-k 16、段落總字數 24,000；語意路由門檻 T=0.58、M=0.04、H=0.85。prompt 模板、每次呼叫的原始回應與重現腳本隨 artifact 提供。

B. RQ0：假想敵——不用工具，LLM 能直接畫出圖嗎

設定。對表 VII 的 7 個系統（有作者參考邊集合的那 7 個），讓 gpt-4.1-mini（temperature 0、JSON 輸出）直接輸出相依邊，與同一份參考邊集合比對，計分規則與 RQ1 相同。三組：（a）**零檔案**：只給 repo 網址與元件清單，模型靠記憶作答；（b）**給檔案**：元件清單加上預算內的部署描述檔、設定檔與原始碼（描述檔 300 KB、原始碼 500 KB、單檔 20 KB；7 個系統中 6 個的原始碼全部給完，TeaStore 給了 162/174 個原始碼檔，Bank of Anthos 的描述檔截在 300 KB）；（c）**給檔案、不給名稱**：同（b），但元件名稱由模型自行決定。元件清單取自參考邊集合，是刻意偏袒 LLM 的設計：它不必猜有哪些節點，只要畫邊。每組每系統執行 3 次，報告平均；表中數字以更正後的參考邊集合重算（第 IV-C 節）。輸出中「X → X」的自迴圈邊（給檔案組 45 條錯誤的邊中有 32 條）顯然是照抄元件清單，表 V 另列一行濾除後的數字。

表 V　純 LLM 直接抽相依圖 VS DEPWEAVER 靜態層（7 個系統、每格 3 次執行的 MACRO 平均）

組別	給 LLM 什麼	P	R	F1	備註
零檔案	repo 網址＋元件清單	0.76	0.68	0.70	7 個系統全部有遺漏的邊，其中 6 個另有錯誤的邊
給檔案	元件清單＋預算內檔案	0.91	0.91	0.90	原始輸出
給檔案（濾除自迴圈）	同上	0.96	0.91	0.93	只拿掉「X → X」的邊；它們都是錯的邊，所以只影響 P
給檔案、不給名稱（嚴格對名）	預算內檔案	0.78	0.77	0.77	名稱要和參考邊集合完全相同；85 個名稱對不上
給檔案、不給名稱（寬鬆對名）	同上	0.97	0.95	0.96	去掉 -container 字尾，或前後綴唯一吻合就算對上
DepWeaver 靜態層（執行 RQ0 當時，14 條規則）	不呼叫 LLM	0.97	0.84	0.88	表 VII「修正前」的 7 列平均；每條邊附出處
DepWeaver 靜態層（依錯誤分析補規則後，23 條）	不呼叫 LLM	1.00	0.99	0.99	表 VII「修正後」；規則在這 7 個系統上歸納，不是 hold-out 數字（hold-out 見表 VIII）
hold-out 5 系統：零檔案	repo 網址＋元件清單	0.51	0.61	0.55	越冷門越差：yas 0.37／0.29、opentelemetry-demo 0.35／0.37
hold-out 5 系統：給檔案	元件清單＋預算內檔案	0.79	0.88	0.83	2 次輸出陷入迴圈（mall-swarm、yas）
hold-out 5 系統：給檔案、不給名稱（嚴格對名）	預算內檔案	0.89	0.86	0.87	這 5 個系統的名稱多半就是 Compose 服務名，對名問題比 7 個歸納用系統小
hold-out 5 系統：DepWeaver 靜態層（46 條）	不呼叫 LLM	0.96	0.93	0.94	表 VIII 最右欄；對這 5 個系統而言不是 hold-out 數字

結果。第一，不給事實時，LLM 畫不出正確的圖：零檔案組平均後沒有一個系統的 P 與 R 同時為 1.00。錯誤的型態有三種：套用別的系統的架構（robot-shop 把負載產生器畫成呼叫 6 個服務；TeaStore 把「全部裝在一個 pod」的部署變體畫成呼叫者）、資料庫接錯（Bank of Anthos 把 ledgerwriter 接到 accounts-db）、方向相反（piggymetrics 4 條）。這 7 個系統都公開在 GitHub 上、很可能在訓練資料裡，對私有系統這一組只會更差。第二，給了檔案之後，LLM 與當時的靜態層各有勝負：在靜態層 recall 較低的三個系統上它都較好（TeaStore 以 enum 常數指定目標的業務呼叫邊，LLM 3 次平均 recall 0.67（三次分別是 1.00、1.00、0.00），靜態層 0/7；ecommerce 只寫了 localhost 預設值的控制平面邊，0.81 對 0.28；robot-shop 的業務呼叫邊，1.00 對 0.40），但在 piggymetrics 較差（recall 0.71 對 1.00；3 次都遺漏的 7 條是 gateway 到 4 個服務、notification 與 statistics 到 auth-service 的業務呼叫邊，以及 turbine 到 rabbitmq）。第三，LLM 的輸出有三個分數看不出來的問題，而且都在這 63 次執行中出現：(i)沒有一條邊附出處，無法像 RQ1 那樣逐條查證；（ii）temperature 0 下同一個 prompt 三次結果不同（TeaStore 給檔案組 recall 0.46–1.00），Bank of Anthos 不給名稱組有兩次陷入迴圈，同樣 12 條邊重複輸出 1,417 次直到輸出上限；(iii)名稱由模型決定時，ecommerce 三次都用 Compose 的 container_name（user-service-container）而不是服務名，嚴格對名下 F1 為 0.00、寬鬆對名下為 0.96。分數的差距幾乎全部是名稱問題，而這正是第 III-F 節請操作者確認的問題。第四，事後逐條追查 LLM 贏過靜態層的那三個系統，靜態層輸的地方沒有一處是靜態分析的極限，都是規則缺口（第 IV-C 節的錯誤分析）；假想敵因此同時是找規則缺口的方法。第五，在 5 個 hold-out 系統上重複同一組對照（表 V 下半，45 次呼叫）：零檔案組 F1 只有 0.55，而且知名度越低越差（yas 0.37／0.29）；給檔案組 0.83，低於 7 個歸納用系統的 0.90，因為 yas 的 17 個模組加 Helm 與 Compose 超過預算、三次都只畫到一半（0.58／0.50）。LLM 唯一明顯勝過靜態層的是 hotelReservation（1.00 對 0.71）：它讀得懂 Go 程式碼裡的 Consul 名稱 srv-search 指的是 search，正是第 IV-C 節所說靜態層真正的邊界。

RQ0 小結。假想敵的答案分兩半：不基於事實的 LLM 畫不出正確的圖；基於事實的 LLM 畫得出來，但給的是一張沒有出處、每次不同、名稱未必對得上的圖。DepWeaver 的貢獻因此不在 P 與 R 本身，而在出處、確定性與名稱確認。反過來，LLM 在 enum 常數指定的目標、隱含預設值與尚未支援文法的語言上的表現，說明它適合作為管線中的一個來源（第 III-B 節已對無文法的語言這樣做），其輸出應進入同一套等級與出處框架（第 V-B 節）。

C. RQ1：靜態層的泛化

參考邊集合一：第三方。MicroDepGraph 資料集 [5] 發表了 20 個開源微服務系統的相依圖，邊來自 docker-compose 中定義的相依關係與 Java 原始碼中的內部 API 呼叫。其中 7 個的 repo 可以直接以靜態層分析，以 Java／Spring 為主（含多語言的 robot-shop）。資料集的相依圖檔提交於 2019-08-19（repo 之後只改過 README），我們把每個 repo checkout 到那一天之前的最後一個 commit，只執行靜態層（不接叢集、不接文件、不呼叫 LLM）；LakesideMutual 的公開歷史始於 2021-02-26（較早的歷史已被壓縮），只能用它最早的 commit。由於兩邊對「相依關係」的定義不同（資料集沒有外部主機，也沒有涵蓋所有程式碼層呼叫，例如 Feign client），這裡報告吻合率（agreement，即資料集的邊中有多少被工具畫出），而非 precision。結果見表 VI。

表 VI　對第三方參考邊集合（MICRODEPGRAPH）的吻合率

系統	資料集邊數	工具畫出	吻合率
spring-petclinic	13	13	1.00
microservices-book	5	5	1.00
tap-and-eat	4	4	1.00
spring-cloud-netflix	26	26	1.00
spring-cloud-microservice	26	24	0.92
LakesideMutual	9	9	1.00
robot-shop	12	12	1.00
合計	95	93	0.98

遺漏的 2 條都在 spring-cloud-microservice：Compose 的 hystrix 服務（Hystrix dashboard）名稱命中函式庫名單，整個節點在正規化時被刪除，這 2 條在 RQ3 由詢問操作者補回。（在先前以 2021-02-26 對齊的版本中，spring-petclinic 另漏 2 條，原因是 hystrix-dashboard 模組在資料集擷取後被移除；改以正確的日期對齊後這 2 條回來了。）工具另外畫了 32 條資料集沒有的邊（外部主機、程式碼層的 Feign 呼叫、反向代理路由、服務註冊與管理元件）；由於無法以第三方來源判定對錯，這些只列出、不計分。

參考邊集合二：作者整理、逐條可複驗。對表 VII 的 7 個系統，作者整理一份參考邊集合（reference edge set），每條邊附第三方可以打開的檔案路徑，並附行號或設定鍵，並以一個自動測試檢查所有檔案出處都存在且內容相符（114 筆；2026-10-26 對各 repo 的固定 commit 重跑）；查核中發現原集合有 2 條邊沒有證據（見錯誤分析），已刪除。另由一個未接觸參考邊集合與工具輸出的 LLM 獨立標註，作為第二位標註者；逐條判定 40 條「只有標註者有」的邊後，沒有一條是作者漏標（17 條為命名差異、5 條為部署變體、18 條為標註者錯誤，其中 3 條方向相反）。結果見表 VII。

表 VII　對作者參考邊集合的 PRECISION 與 RECALL

系統	畫出	正確	錯誤	遺漏	P	R	修正後 P	修正後 R
Bank of Anthos	12	12	0	0	1.00	1.00	1.00	1.00
ecommerce	20	20	0	17	1.00	0.54	1.00	1.00
ewolff-k8s	5	5	0	0	1.00	1.00	1.00	1.00
Online Boutique	17	17	0	0	1.00	1.00	1.00	1.00
piggymetrics	41	33	8	0	0.80	1.00	1.00	1.00
robot-shop	18	18	0	3	1.00	0.86	1.00	0.90
TeaStore	6	6	0	7	1.00	0.46	1.00	1.00

「修正前」是執行 RQ0 時的 14 條規則；「修正後」是依下文錯誤分析補上第 15 至 23 條之後。修正後只有 robot-shop 仍有遺漏（Node 與 PHP 各 1 條）。

錯誤分析（修正前）。每一個非 1.00 的數字都能追溯到具體原因；而且逐條追查後，先前歸因於「靜態分析的極限」的三處，都是規則缺口。ecommerce 遺漏的 17 條全是控制平面：8 條到設定中心，repo 只寫了 optional:configserver:http://localhost:9296，而 cloud-config 的 server.port 正是 9296；9 條到服務註冊中心，repo 內外都沒有任何 eureka.client 設定，只有 Eureka client starter 與 @EnableEurekaClient，位址是 Spring Cloud 的隱含預設 localhost:8761。工具依「localhost 不是節點」丟掉它們，沒有編造，業務呼叫邊 P、R 皆 1.00（原參考邊集合另有 service-discovery → cloud-config 一條，查核時發現 service-discovery 沒有 config client 依賴，已刪除）。piggymetrics 的 8 條錯誤的邊來自同一個原因：共用 application.yml 中的 OAuth、RabbitMQ 與 Eureka 設定被攤到每一個 config client，實際上只有宣告了對應函式庫的服務會使用；其中 monitoring → registry 原本列在參考邊集合中，查核 monitoring 的建置檔沒有 Eureka client 後，改列為錯誤的邊並自參考邊集合刪除。TeaStore 遺漏的 7 條業務呼叫邊，呼叫都以 Service.PERSISTENCE 這類 enum 常數指定目標；但這不是動態分派：常數的字串值 tools.descartes.teastore.persistence 就是模組名，registry 在執行期只決定實例的位址，不決定目標是誰，靜態上可以解析。robot-shop 遺漏的 3 條中，2 條位於尚未支援文法的 Node 與 PHP 程式碼；第 3 條 shipping → cart 是 Java 的 String.format 加上 getenv("CART_ENDPOINT", "cart")，預設值與 Dockerfile 的 ENV 都可以靜態取得，同樣的寫法在 Python 已經會處理。

規則的來源。第一輪（修改前）6 個新系統全部有問題：TeaStore 整張圖縮減為 3 個節點（模組名以「.」分隔，被當成 DNS 後綴切掉）；ecommerce 近半的邊來自常數檔（56 條中的 25 條，其中 19 條的常數沒有被引用），另有 Feign contextId 造成的偽節點；piggymetrics 的資料層全部遺漏（設定位於 Config repo）。逐條比對第一輪的差異，歸納出 10 條規則；修改後 Bank of Anthos 既有圖的邊逐條不變，train-ticket 的既有邊全部保留。第三方對照又歸納出 3 條（第 11–13 條）；追查 Online Boutique 為什麼 recall 只有 2/16（約 0.13；參考邊集合後來補上一條，現為 17 條）時，再歸納出第 14 條「workload 本身環境變數中的服務位址畫成僅推測」，使其 recall 提高到 1.00。原本歸因於「語言尚未支援文法」的損失，大半是部署描述檔已經寫明、但工具沒有畫出的宣告。

第二輪：由錯誤分析補規則，並以 hold-out 量測轉移。依上述錯誤分析再歸納 9 條（第 15 至 23 條）：localhost:PORT 讀回唯一宣告該 port 的模組、Eureka 的隱含預設、字串值等於模組名的常數、String.format 與 getenv 的預設值，以及在私有系統 AICS 上逼出的 5 條（Deployment 的版本字尾、環境變數依讀取者的範圍解析、函式庫模組不是節點、Python 的宣告不等於使用、共用設定只在有對應函式庫的 client 生效）。修正後 7 個系統 precision 全為 1.00，recall 除 robot-shop（0.90）外皆 1.00（表 VII「修正後」）；Bank of Anthos 與 train-ticket 逐邊不變，第三方吻合率維持 93/95；AICS 由 P 0.42、R 0.10 提高到 0.89、0.63。但這些數字不是 hold-out：規則就是在這些系統上歸納的。為了量測規則對未見過系統的轉移，我們在歸納前另外選定 3 個不在任何歸納集合中的系統（表 IV 的 hold-out B），先寫好參考邊集合並記錄修正前的分數，再以修正後的工具重跑：**三個系統的邊一條都沒有改**

變（表 VIII）。第 15 至 23 條描述的是 Spring Cloud 與 Python 的慣例，hold-out B 的錯與漏來自另一批寫法：Compose 只寫鍵名、值在 .env 的環境變數轉發；links 與 external_links 的別名；OpenAPI 中繼資料的 URL 被當成呼叫；Spring Cloud Alibaba 的 server-addr；Envoy 設定；Kotlin。

第三輪：從 hold-out B 歸納，以 hold-out C 量測。依 hold-out B 的差異再歸納 12 條（第 24 至 35 條），hold-out B 由 P 0.73、R 0.70 提高到 0.99、0.89（hotelReservation 的 9 條遺漏是 Go 經 Consul 以 srv-* 名稱解析、程式碼與部署描述檔都沒有主機字串，維持不畫）；7 個歸納用系統逐邊不變，train-ticket 少了一條 Swagger termsOfServiceUrl 造成的假外部邊。這一輪的轉移以再另外選定的 2 個系統量測（hold-out C）：yas 由 P 0.58、R 0.58 提高到 0.68、0.81，業務呼叫邊 recall 由 0.11 提高到 0.89，轉移的是 Compose 環境變數轉發那一條規則；run-aspnetcore-microservices 維持 0.00、0.00，因為它暴露的是另一個缺陷：Compose 服務名含「.」（catalog.api）被當成 DNS 後綴截斷。依 hold-out C 再歸納 11 條（第 36 至 46 條）後，hold-out C 為 1.00／1.00 與 0.83／0.96，hold-out B 不變；但這兩個數字已不是 hold-out 數字。

表 VIII　規則對未見過系統的轉移。每一欄是一輪規則加入後的分數；粗體格是真正的 HOLD-OUT 量測（規則在上一組系統上歸納，這一組在歸納前就選定、沒有看過）

系統	指標	14 條	第 15–23 條後	第 24–35 條後	第 36–46 條後
opentelemetry-demo（B）	P	0.79	**0.79**	0.97	0.97
	R	0.82	**0.82**	1.00	1.00
mall-swarm（B）	P	0.43	**0.43**	1.00	1.00
	R	0.50	**0.50**	0.96	0.96
hotelReservation（B）	P	1.00	**1.00**	1.00	1.00
	R	0.71	**0.71**	0.71	0.71
yas（C）	P	—	0.58	**0.68**	0.83
	R	—	0.58	**0.81**	0.96
run-aspnetcore-microservices（C）	P	—	0.00	**0.00**	1.00
	R	—	0.00	**0.00**	1.00

三輪的觀察一致：每一組未見過的系統都暴露出工具沒學過的寫法；修正後，先前各組的分數都不退步。所以靜態層的 recall 不是靜態分析的極限，而是寫法涵蓋率；對一個未見過的系統，應預期 recall 落在各組 hold-out 修正前的水準（0.5 至 0.8），precision 的損失則集中在少數可以通用修正的錯誤類型（中繼資料 URL、別名、測試 fixture、名稱截斷）。

RQ1 小結。靜態層的 precision 高：歸納用的 7 個系統修正後全為 1.00，第三方吻合率 0.98，hold-out B 修正後 0.99。recall 取決於寫法涵蓋率，三輪 hold-out 顯示每組新系統都會暴露新寫法。真正需要執行期觀測補足的邊界剩下兩類：沒有文法的語言（Node、PHP、Go 只有薄表面），以及目標只以 registry 名稱存在、程式碼與部署描述檔都沒有主機字串的呼叫（hotelReservation 的 9 條 Consul srv-*）。

D. RQ2：融合執行期觀測

實驗設定。Bank of Anthos 部署於 k3s 與 Istio 的叢集，含真實的 PostgreSQL 資料庫（accounts-db、ledger-db）。DepWeaver 以完整流程執行：收集、產生流量、查詢量測指標、缺口補齊、產出。圖 3 是產出的分層相依圖。

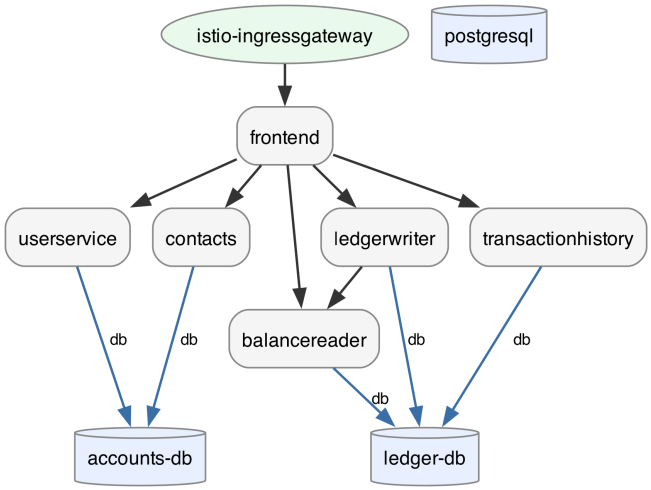


圖 3. Bank of Anthos 的分層相依圖（DepWeaver 輸出，版本標籤省略）。postgresql 是一個沒有任何邊的泛稱節點（其來源未能確定），不列入計分

結果。業務呼叫覆蓋率 7/7，資料層 5/5。在加入 TCP 查詢之前，資料庫邊沒有任何執行期證據：Istio 只對 HTTP 與 gRPC 產生請求指標，資料庫連線是不透明的 TCP；加入服務網格內的 TCP 連線查詢後，資料庫邊才有執行期證據。

同一條邊的三種狀態。表 VIII 是 transactionhistory 到 ledger-db 這條邊在三次實驗中的狀態。

表 IX　TRANSACTIONHISTORY → LEDGER-DB 在三次實驗中的狀態

實驗	模型中的狀態	原因
靜態（修正前）	不存在	資料來源由環境變數隱式綁定，程式碼中沒有讀取的行
靜態（修正後）	虛線：有使用證據，未在執行期量測到	從 Deployment 的 envFrom 讀到注入的位址，加上 JPA 持久化 annotation
執行期	實線：量測到 5,191 次連線	服務網格內的 TCP 查詢

等級影響下游的回答。我們以同一組 10 個問題、同一個模型（gpt-4.1-mini），在三次實驗的存檔上問答，比較三組：只給報告、報告加檢索、給圖查詢結果（DepWeaver）。在修正前靜態存檔上，圖遺漏了上述那條邊，三組都依圖回答沒有這條關係，與事實不符；在執行期存檔上，三組都答對，DepWeaver 組的回答是「量測到」；在修正後靜態存檔上也都答對，DepWeaver 組的回答是「宣告且有使用證據，但未在執行期量測到」。由於標準答案由同一張圖的查詢產生，這組對照衡量的是回答與圖一致的程度，而非與事實一致的程度。結果顯示，問答的正確性受限於圖的正確性，而圖的錯誤會原樣傳到回答裡。

E. RQ3：對話式缺口補齊

流量參數（Bank of Anthos）。為了確認第 3 層確實是最後手段，我們關閉第 1 層再執行一次。工具自動觸發到 5/7 後無法繼續推進：剩下兩條（包含 ledgerwriter 到 balancereader）需要一筆實際寫入成功的轉帳，也就是需要這個部署的密碼，以及另一位使用者的收款帳號。工具在 Discord 詢問這兩個值，操作者回答後自動重跑，覆蓋率到 6/7；第一次提供的收款帳號是匯款人自己的帳號，轉帳不會觸發餘額檢查；產生器當時無法重問，只能把該邊宣告為無法觸發。修改 prompt、允許以新的變數名重問之後，換成他人帳號，覆蓋率到 7/7。這次執行（2026-08-26）的依據是當時對話中的覆蓋率訊息與操作者紀錄，分析存檔沒有保留下來；RQ2 的五次執行期分析與 RQ3 的名稱實驗則都有完整存檔（第 III-G 節）。

名稱（spring-cloud-microservice-examples）。RQ1 的第三方對照中，這個系統遺漏了 hystrix 到 discovery 與 hystrix 到 gateway 兩條邊。根因是 Compose 的 hystrix 服務（Hystrix dashboard）名稱命中函式庫名單，整個節點在正規化時被刪除。改為「名稱命中函式庫名單、卻有來自程式碼或部署層的邊時提問」之後，工具提出 3 個問題，回答後吻合率由 24/26 提高到 26/26，其他 6 個系統的結果逐條不變。

在 Discord 上的五次實際執行。同一個系統在實際的 Discord 環境中執行了五次，每一次都暴露介面或規則的問題並修正：表單只接受候選時，加入完整的詞彙表；同一服務的模組名與部署名重複出現時，改問「是否為同一個」；文件把 repo 名稱當成元件時，直接丟棄不問。從第二次起，圖穩定在 26/26；從第三次起，先前保存的答案開始生效：先前答過的 11 個名稱不再詢問，只詢問新出現的 5 個；第四、五次分別詢問 4 個（其中 2 個由程式錯誤造成）與 2 個。五次中有四次留下完整存檔，報告末節累計套用的操作者答案依序為 8、11、16、20 個，與上述詢問次數相符。我們也觀察到 DeepWiki 每次的用詞不同，因此每次執行都可能零星出現需要詢問的新名稱。

限制。五次實際執行與離線驗證的操作者都是作者本人，而且看過參考邊集合。這只顯示機制在實際環境中可以運作，無法說明不熟悉系統的操作者能否答對。

F. RQ4：設計規則防止的錯誤

表 IX 的每一個錯誤都在開發與驗證中實際發生過，右欄是防止它的規則（第 III-C、III-E 節）。

表 X　實際發生過的錯誤與防止它的規則

發生的錯誤	防止它的規則
petclinic：覆蓋率被控制平面、框架產生的偽節點與別名稀釋成 45%；只算可觸發的業務呼叫邊後為 4/4	只計可量測關係的覆蓋率分母
Bank of Anthos：同一叢集、同一段流量，文件多提了幾條不存在的邊，覆蓋率 7/7 變成 7/10	只計可量測關係的覆蓋率分母＋信心等級
Bank of Anthos：行程內快取永遠無法量測，資料層卡在 5/6，覆蓋率訊息還暗示重啟 pod	只計可量測關係的覆蓋率分母（行程內元件）
Bank of Anthos：報告把資料庫邊寫成「執行期是否觀測：未知」，旁邊的圖畫實線；改 prompt 兩次，第二次把建置期函式庫列為相依關係	一個模型、多個視圖＋事實由程式產生
spring-cloud-microservice：LLM 撰寫的報告列出操作者已回答 ignore 的名稱	事實由程式產生
無叢集模式的報告寫出「25 Services observed、50 Pods Running」	未量測不是零
無叢集模式的覆蓋率訊息把未量測寫成「observed 0/23（0%）」	未量測不是零
容器映像檔中沒有 git，程式碼抽取從未成功，流程照樣完成、報告看似正常	未量測不是零（失敗留下紀錄）

表中第 1 列的數字來自 2026 年 7 月的對話紀錄，該次分析存檔未保留；其餘各列都有對應的 commit、分析存檔或測試可查。八個錯誤中，多數不是量測本身的錯誤，而是在證據轉換為讀者所見產出物的過程中被放大或掩蓋。在第 4 與第 5 個錯誤上，修改 prompt 後錯誤以另一種形式出現；改由程式產生該段落之後，後續執行中沒有再觀察到同類錯誤。

V. 討論

A. 適用範圍

DepWeaver 的設計假設三件事：證據來自可靠度不同的多個來源；產出物不只一種（圖、數字、報告）；讀者會依據產出物做決策。只有單一來源時（例如純追蹤產生的服務圖），信心分級沒有作用。

B. 靜態與執行期的分工

RQ1 與 RQ2 顯示兩層互補而非互相替代：靜態層 precision 高，但 recall 受寫法涵蓋率限制，每組未見過的系統都會暴露新寫法（第 IV-C 節）；只以 registry 名稱解析、沒有任何主機字串的呼叫（hotelReservation 的 9 條）則是它真正的邊界。執行期觀測精確，但只看得到被觸發的路徑，需要產生流量與補齊缺口才能看到更多。piggymetrics 的 8 條錯誤的邊原本被當成另一個方向的互補（靜態層分不出共用設定誰在用，要靠執行期觀測以「未量測到」降級）；第 23 條規則改以建置檔的相依套件判斷後，靜態層就分得出來，執行期觀測退為第二道防線。RQ0 補上第三個方向：給了檔案的 LLM 讀得懂 enum 常數、隱含預設值與尚未支援文法的語言，正好落在規則缺口上，假想敵因此也是找規則缺口的方法；但它的輸出沒有出處、每次不同。合理的分工不是以 LLM 取代規則，而是把 LLM 的抽

取結果當成一個來源併入融合模型，等級不高於「有使用證據」、出處記為「LLM 自某檔案推得」，名稱走同一個確認流程。

C. 為什麼最後才詢問操作者

詢問操作者有成本：會打斷操作者，而且操作者可能答錯。所以 DepWeaver 只在前面的自動手段失敗時才問，而且問題必須具體到可以用一個值或一個選項回答。唯一的例外是登入憑證：猜錯的密碼不會明顯失敗，因此缺少時直接整組詢問。RQ3 的兩個情境都符合這個條件：密碼與收款帳號屬於部署的事實，在程式碼中找不到；一個名稱指哪個服務，是只有熟悉系統的人知道的對應。

D. 效度威脅

建構效度：作者參考邊集合由作者整理；我們以第三方資料集、逐條可複驗的出處與第二位標註者降低這個風險；不過第二位標註者是 LLM 而非人類，無法完全取代獨立的人工標註。此外，第三方資料集對「相依關係」的定義與本工具不同，所以只報告吻合率。**內部效度：**RQ3 的流量參數實驗只有當時的對話紀錄，沒有可重讀的分析存檔，5/7、6/7、7/7 三個數字無法在事後由第三方重算；46 條抽取與合併規則都是在受測系統上歸納出來的；表 VII「修正後」與各組 hold-out 修正後的數字都不是 hold-out 結果，只有表 VIII 中粗體的格子是。每輪新增規則後，先前所有系統的節點與邊都逐項重算，沒有退步（唯一的邊變動是 train-ticket 少了一條 Swagger 中繼資料造成的假邊）。hold-out 的參考邊集合是在工具執行前寫好的，但同樣由作者整理。另外，RQ3 的操作者是作者本人且熟悉受測系統，可能高估名稱問題的回答正確率。**外部效度：**歸納用的系統以 Java 與 Spring 為主；hold-out 加入了 Go、.NET、Kotlin、Rust 等語言，但這些語言只有薄表面抽取。含缺口補齊的完整執行期驗證只有 Bank of Anthos 一個案例。RQ0 的假想敵在 7 個歸納用系統與 5 個 hold-out 系統上各執行一次，但 hold-out 那一組比較的是第三輪規則後的靜態層。**RQ0 的對照：**只用一個模型（gpt-4.1-mini）、每格 3 次；元件清單取自參考邊集合，偏袒 LLM；檔案預算由我們決定（但 6 個系統的原始碼全部給完，有截斷的 TeaStore 與 Bank of Anthos 的 recall 已達 0.82 與 1.00，預算不是瓶頸）。**LLM 輸出的變異：**報告與文件整理倚賴 LLM，DeepWiki 的用詞每次不同；事實段落由程式產生而不受影響，LLM 撰寫的章節與文件來源的邊則會受影響。

VI. 結論與未來工作

本文提出 DepWeaver，以三來源融合、三級信心等級與逐條出處建構微服務相依圖，讓圖、覆蓋率與報告由同一個模型產生而彼此一致，並在自動手段用盡時才於 ChatOps 對話中詢問操作者。在 19 個開源系統與 1 個私有系統上，靜態層對第三方參考邊集合的吻合率為 0.98；三輪「歸納、hold-out」的評估顯示規則的轉移是部分的、每組未見過的系統都暴露新寫法，靜態層的 recall 是寫法涵蓋率而非分析的極限；作為假想敵，不給任何檔案的 LLM 直接畫圖 F1 為 0.70，給了 repo 檔案後為 0.90，但沒有出處、三次執行結果不同、自行決定的名稱有 85 個對不上；在一個真實部署（Bank of Anthos）上，融合執行期觀測後業務呼叫與資料層覆蓋率都達到 100%；關閉 repo 範例來源時，最後兩條業務邊由對話補齊。未來工作包括：請不熟悉受測系統的操作者回答名稱問題，量測詢問操作者的實際正確率；在 TeaStore 上完成執行期驗證，量化靜態與執行期的互補；以小型使用者研究檢驗信心分級是否改善讀者的決策；以及以相依圖產生部署順序，並以執行期證據驗證。

VII. 附錄：46 條抽取與合併規則

表 X 是第 III-D 節所述規則的全表（第 1 至 14 條為初稿時的規則，第 15 至 23 條為第二輪新增，第 24 至 35 條為三個 hold-out 系統逼出的第三輪，第 36 至 46 條為第二組 hold-out 兩個系統逼出的第四輪）。每一條都是在某個受測系統上追查差異時歸納出來的，沒有一條寫死專案名；「觸發」欄是第一次逼出這條規則的系統，「位置」欄是實作它的元件。

表 XI　46 條抽取與合併規則（全表）

#	規則	觸發	位置
1	manifest 的 workload 名稱（Deployment、StatefulSet、DaemonSet、Job）是無叢集模式的服務名稱清單；模組目錄以完全相同或「-部署名」結尾對上 workload；對不上的模組（函式庫、基底映像）不預先當節點，有邊碰到才出現	ewolff、TeaStore、Online Boutique	Config Extractor、CodeGraphMerger
2	模組目錄名正規化：反向網域名（tools.descartes.teastore.webui）的「.」換成「-」；src、app 這類包裝目錄往上找；*-tests、e2e 不是服務；含兩個以上子模組的父目錄是聚合，不是服務	TeaStore、Online Boutique、ewolff	ServiceRootScanner
3	Spring Cloud Config 倉庫版面：不叫 application 的 Spring 形狀 yml 也讀，檔名對到服務就歸該服務；旁邊的共用 application.yml 攤到每個 config client	piggymetrics	Config Extractor、CodeGraphMerger
4	佔位符依序查環境變數、屬性表、${key:default} 的預設值，最後才用變數名本身（如 ${CATALOGUE_HOST} → catalogue），這一步標為僅推測	piggymetrics、ewolff、robot-shop	CodeGraphMerger
5	裸主機名也是目標：spring.data.mongodb.host、zuul.routes.x.serviceId、DB_HOST 這類沒有 port 的值，變數名已說明它是主機	piggymetrics、TeaStore、robot-shop	Config Extractor
6	反向代理設定是入口的邊清單：Apache ProxyPass、nginx proxy_pass 的上游視為入口的呼叫	ewolff、robot-shop	Config Extractor
7	Compose 描述的是服務之間的連線設定，不是某個服務的設定：services.<x>.environment 進環境變數表；depends_on 的目標也建立節點；Compose 名對 workload 以去掉 -container 字尾或唯一字尾對應	TeaStore、robot-shop、ecommerce	Config Extractor、CodeGraphMerger
8	不是目標的東西不畫：localhost 與 127.0.0.1 永遠不是節點；Feign 的 contextId、path、fallback 是 bean 設定；Feign 宣告了 url 時 name 只是 bean id；值不是位址的設定列不算「解不開」	ecommerce、piggymetrics、TeaStore	CodeGraphMerger
9	宣告不等於使用：static final String 的 URL 常數若在自己模組內沒有任何引用就不畫邊；@Entity、@Table 屬 Java 標準（JPA）而非 Spring；JAX-RS 的 @Path 視為 HTTP 伺服端	ecommerce、TeaStore	TreeSitterExtractor、查詢檔
10	只認「部署了這個 repo 自己模組」的 manifest 目錄：repo 附帶的監控堆疊與選配元件（prometheus、grafana、jaeger、pgpool operator…）不進名稱清單；一個目錄要有至少一個 workload 對得上 repo 的模組才算	train-ticket、Bank of Anthos	Config Extractor
11	Compose 的 services 也是部署名稱清單，與 Kubernetes workload 同級；但有 manifest 時完全不看 Compose 名，避免同一服務出現兩種拼法	spring-petclinic、microservices-book、Tap-And-Eat、spring-cloud-microservice	Config Extractor、CodeGraphMerger

#	規則	觸發	位置
12	也認 Compose 第 1 版格式（沒有 services:，頂層即服務，須有 image 或 build）與 links:（服務:別名，取冒號前）	Tap-And-Eat、spring-cloud-microservice	ConfigExtractor
13	XML namespace 不是位址：URL 字面值所在行或前兩行出現 namespace、schemaLocation、xmlns、soapAction 時不畫邊	LakesideMutual	TreeSitterExtractor
14	workload 自己環境變數裡的服務位址是宣告，畫成僅推測；共用 ConfigMap 注入的仍不畫（它會被發給不呼叫它們的 workload）；位址對節點用寬鬆比對（manifest 常注入短名 registry，workload 叫 teastore-registry）	Online Boutique、Bank of Anthos、TeaStore、LakesideMutual	CodeGraphMerger
15	loopback 位址以 port 讀回擁有者：client 宣告中的 localhost:PORT，若恰好一個模組的 server.port 是 PORT，畫到該模組並標為僅推測；零個或多個擁有者就不畫，localhost 仍不是節點；Spring 的 configserver: 匯入前綴先剝掉	ecommerce	ConfigExtractor、CodeGraphMerger
16	Spring Cloud 的隱含註冊中心：建置檔有 Eureka client starter、設定中沒有 defaultZone 的模組，連到唯一有 Eureka server starter 的模組，標為僅推測；零個或多個 server 就略過；兩端都要在部署名稱清單中	ecommerce	ServiceRootScanner、CodeGraphMerger
17	字串值等於模組名的常數就是被呼叫端：enum 常數或 static final String 的值正規化後等於某模組目錄名，而 Type.NAME 在可部署模組的呼叫點當參數出現，就畫到該服務（有使用證據）；自我引用與共用 client 函式庫中的引用不畫	TeaStore	查詢檔、CodeGraphMerger
18	String.format("http://%s/…", getenv("X", "d"))：%s 先以模組自己的環境（Dockerfile ENV、manifest env）填入，再用 getenv 的預設值；Dockerfile 的 ENV 列進環境變數表，範圍是該 Dockerfile 的模組	robot-shop	查詢檔、ConfigExtractor
19	帶版本字尾的 Deployment 是某個 Service 的一個版本：X-v<數字> 旁有同名 Service X、或有另一個只差版本的 workload 時，節點名用 X；只剝 -v<數字>，且只在這兩種證據下	AICS（私有系統）	ConfigExtractor、CodeGraphMerger
20	環境變數在讀取者自己的範圍解析：模組 X 的 getenv 先查 X 的 workload 自己的 env、X 收到的 ConfigMap、X 的 Dockerfile，都沒有才查全 repo 表，且只在值一致（或恰有一個是已知服務）時採用；範圍內多個不同值視為模糊、不畫	AICS	CodeGraphMerger
21	函式庫不是服務：只有 setup.py 或 pyproject.toml、沒有 Dockerfile 的 Python 模組不成為節點；其檔案中的呼叫歸給 import 該檔案（含函式庫內部相對 import 的傳遞）的服務；沒人 import 就不畫	AICS	ServiceRootScanner、查詢檔、CodeGraphMerger

#	規則	觸發	位置
22	呼叫點才算使用（Python）：變數 URL 沿同檔賦值追到 getenv 或 URL 字面值，邊的出處是發出請求的那一行；模組中沒有任何活程式碼引用的函式（含只被死函式呼叫者）內的呼叫，與沒人引用的 NAME = os.getenv(…)，視為宣告、不畫邊	AICS	查詢檔、TreeSitterExtractor
23	共用設定的 key 只在有對應函式庫的 client 生效：spring.rabbitmq.* 只在宣告 AMQP artifact 的模組綁定，security.oauth2.* 要有 oauth2、eureka.client.* 要有 eureka，kafka、mongodb、redis 同理；讀不到建置檔的模組一律保留	piggymetrics	CodeGraphMerger
24	Compose 的鍵名轉發與 .env 內插：environment 裡只寫鍵名的項目（-CART_ADDR）是從專案 .env 或該服務 env_file 轉發的變數，值照 Compose 規則內插（${VAR}、${VAR:-d}，可遞迴）；env_file 的每個鍵都是該服務的環境變數	opentelemetry-demo	ConfigExtractor
25	links 與 external_links 的別名是該服務對目標的稱呼：mysql:db 讓這個容器以 db 連到 mysql，其設定寫 jdbc:mysql://db；別名限該服務的範圍，解析時先查讀取者自己宣告的，沒有才用全 repo 一致的那個	mall-swarm	ConfigExtractor、CodeGraphMerger
26	連線 scheme 決定節點種類：jdbc:、r2dbc:、mongodb://、redis:// 一律是資料庫，amqp://、kafka:// 是 broker，不論主機叫什麼	mall-swarm	CodeGraphMerger
27	位址清單 key 與裸 FQDN 的 endpoint：server-addr、uris、cluster-nodes、addresses、bootstrap-servers 的值是 host:port（無 scheme），列為目標，清單取第一個；*.endpoint 的值若是帶點的裸主機名是外部服務，裸單字仍只是訊號	mall-swarm	ConfigExtractor、CodeGraphMerger
28	Kotlin 以薄表面讀：.kt 視為 JVM 的確定性 stack，但沒有離線的 tree-sitter 文法，改以正規表示式讀 getenv、URL 字面值與 listener 註解，並在輸出中寫明看不到變數主機的呼叫點	opentelemetry-demo	StackDetector、TreeSitterExtractor
29	OpenAPI／Swagger 中繼資料的 URL 不是呼叫：字面值是 License、ExternalDocumentation、Contact、Info、Server 鏈上 url()、termsOfService()，或 ApiInfoBuilder、Contact 建構子、@Info 等註解 url 屬性的參數時，不畫邊；判斷的是 builder，不是網域名單	mall-swarm、train-ticket	查詢檔、TreeSitterExtractor
30	訊息程式碼的邊是服務 → broker：@RabbitListener、@KafkaListener、kafkaTemplate.send 的目的地名稱不是節點；broker 是該服務設定裡的那一個（已連到的同族 queue 節點，否則部署中唯一的同族 queue 節點），都沒有就當殘餘；consumer 與 producer 同向	mall-swarm	CodeGraphMerger

#	規則	觸發	位置
31	測試 fixture 不是服務（規則 2 的延伸）：檔名片段含 test、tests、e2e 的 Compose 檔整份不讀；服務名以 tests 結尾或符合測試目錄規則的不建節點；隱藏目錄（.bingo）裡的建置標記是工具	opentelemetry-demo、hotelReservation	ConfigExtractor、ServiceRootScanner
32	key=value; 連線字串的主機：ADO.NET、Npgsql 的 Host=…;Username=… 與 Server=…;Database=… 取 Host、Server、Data Source 當位址	opentelemetry-demo	ConfigExtractor
33	保留 TLD 不是相依：RFC 2606／6761 的 .test、.example、.invalid、.localhost、example.com 與 RFC 6762 的 .local 結尾的主機不畫邊、不留殘餘、不出 ServiceEntry	opentelemetry-demo	CodeGraphMerger、ExternalHostDetector
34	Envoy 靜態設定是反向代理的上游表（規則 6 的延伸）：static_resources.clusters 裡每個 socket_address 出一條 url 列，port 是佔位符時省略	opentelemetry-demo	ConfigExtractor
35	depends_on 是啟動順序，分級處理：宣告它的服務自己的環境變數沒有任何位址時照畫、標為僅推測；有位址時，有位址背書的 depends_on 標為有文件證據，沒有位址背書的不畫。17 個系統的 100 條 depends_on-only 邊：90 對、5 variant、5 錯，5 條錯全來自環境變數有位址的服務	opentelemetry-demo（錯邊）、MicroDepGraph 7 系統（保住 90 條）	CodeGraphMerger
36	只剝真正的 DNS 字尾：叢集字尾（.svc.cluster.local、.svc）連同前一段 namespace 一起剝；<svc>.<ns> 只在 ns 是 manifest 宣告過的 namespace 時剝；等於部署宣告過的帶點名字（compose 服務 catalog.api）整個保留，Basket.API 目錄與 basket.api 以模組拼法視為同名；保留 TLD 在任何截斷之前判斷；其餘帶點主機是外部網域，永遠不截到第一段	run-aspnetcore-microservices、yas	ConfigExtractor、CodeGraphMerger
37	Compose 的 hostname: 是該服務的全域別名（規則 25 的延伸）：hostname: ecommerce-mq 讓 amqp://ecommerce-mq 指向宣告它的服務，別名本身不是節點	run-aspnetcore-microservices	ConfigExtractor、CodeGraphMerger
38	YARP 的 ReverseProxy.Clusters.*.Destinations.*.Address 是閘道的上游表（規則 6、34 的延伸）；appsettings*.json 其餘以冒號串起的鍵進設定列，與 compose 以雙底線拼寫的同一鍵等同	run-aspnetcore-microservices	ConfigExtractor
39	值的形狀決定節點種類（規則 26 的延伸）：key=value; 連線字串與 ConnectionStrings__X 一律是資料庫、amqp:// 是 broker，不論主機叫什麼；先畫成呼叫的邊在目標種類確定後改成 db／async	run-aspnetcore-microservices	ConfigExtractor、CodeGraphMerger

#	規則	觸發	位置
40	C# 以薄表面讀（規則 28 的延伸）：GetConnectionString("X")、Configuration["A:B"]、GetValue、GetSection 的鍵換成環境變數拼法 A__B，GetEnvironmentVariable 與 URL 字面值照讀；鍵在 compose 的 env 表解到主機就是有文件證據的呼叫點；輸出寫明看不到變數主機的呼叫點	run-aspnetcore-microservices	StackDetector、TreeSitterExtractor
41	JVM 函式庫模組不是服務（規則 21 的 Java 版）：沒有 Dockerfile、沒有 Boot／shade／assembly／jib 外掛、不是 war、src/main 下沒有 main 的 pom／gradle 模組是函式庫，其設定檔不畫邊、不成節點；部署檔點名它時以部署為準	yas	ServiceRootScanner、CodeGraphMerger
42	測試樹的每一段都算（規則 2、31 的延伸）：src/it、src/integration-test、e2e、__tests__、automation-ui 底下的設定與程式碼不讀；模組路徑任何一段符合測試目錄規則就不是服務根	yas	SourceScanner、ServiceRootScanner
43	OpenAPI UI 的設定鍵不是呼叫（規則 29 的設定檔版）：springdoc.、*springfox.*、swagger.、*knife4j.* 底下的 URL 是瀏覽器端 Swagger UI 要開的網址	yas	ConfigExtractor
44	compose 的 build 說了哪個模組是哪個服務：build 的 context（或 dockerfile 所在目錄）對到的模組目錄就叫那個 compose 服務名，模組自己的 artifactId 等於 compose 服務名亦同；是 repo 寫的對應，不是相似度，不問人；有 k8s manifest 時 compose 名要能對到 workload 才採用	yas、run-aspnetcore-microservices、ewolff/microservice	ConfigExtractor、ServiceRootScanner、CodeGraphMerger
45	*.url*／.uri／*.endpoint 鍵下的裸單字只在它是部署宣告過的名字（詞彙表或 compose 服務名）時才是目標（規則 5、27 的延伸）：elasticsearch.url=elasticsearch 畫到 elasticsearch，feature.url=disabled 不畫	yas	CodeGraphMerger
46	nginx 的 set $var "x"; proxy_pass http://$var：同檔內設成字面值的變數填回 proxy_pass（一層），沒設的仍是佔位符	yas	ConfigExtractor

第 1 至 9 條來自六個新系統的第一輪差異（第 IV-C 節「規則的來源」），第 10 條來自 Bank of Anthos 與 train-ticket 的回歸測試，第 11 至 13 條來自第三方參考邊集合的對照，第 14 條來自追查 Online Boutique 的 recall。第 15 至 23 條來自第二輪：先對七個系統剩下的漏邊與錯邊逐條追到形狀（第 15 至 18、23 條），再以一個未曾見過的私有 Istio 多 repo 系統（13 個 repo，Python）當對照（第 19 至 22 條）。每一條新增後，既有系統的節點與邊都逐項重算，確認沒有退步：Bank of Anthos 與 train-ticket 逐邊不變，第三方參考邊集合的吻合率維持 93/95。第 24 至 35 條來自第三輪：先把規則 1 至 23 原封不動地跑在三個未曾見過的開源系統上（opentelemetry-demo、mall-swarm、DeathStarBench hotelReservation；作者另行整理參考邊集合），得到 P 0.73、R 0.70——前兩輪的規則沒有一條轉移過去；再對 24 條錯邊與 28 條漏邊逐條追到形狀。新增後三系統合計 P 0.99、R 0.89（hotelReservation 剩下的 9 條漏邊是 Go 經 Consul 註冊名解析的呼叫，程式碼與部署檔都沒有主機名，是已記錄的邊界），但這個數字已不是 hold-out 結果：規則是在這三個系統上歸納的。既有系統的回歸同前：Bank of Anthos 逐邊不變、train-ticket 少一條 Swagger 中繼資料造成的假外部邊、第三方吻合率維持 93/95、七個歸納系統的 precision 全部維持 1.00。第 36 至 46 條來自第四輪：規則 1 至 35 原封不動地跑在第二組未曾見過的系統上（nashtech-garage/yas，Spring Boot 3 ＋ BFF ＋ Keycloak ＋ Kafka；aspnetrun/run-aspnetcore-microservices，.NET 8 全 C#），得到 yas P 0.68、R 0.81，.NET 系統 P 0.00、R 0.00——後者 13 條畫出的邊有 11 條只差在名字：compose 服務叫 catalog.api，工具把含點的名字都在第一個點截斷。逐條追到形狀後新增 11 條，之後 .NET 系統 P 1.00、R 1.00，yas P 0.83、R 0.96（剩下的錯邊是 compose 把位址交給了不呼叫它的服務，與瀏覽器導回網址；剩下的漏邊是 Debezium connector 設定、Next.js SSR 與 nginx 的 server_name，均記為邊界）。回歸：七個歸納系統邊逐條相同、Bank of Anthos 與 train-ticket 逐邊不變、第三方吻合率 93/95 不變（ewolff/microservice 工具多畫的兩種拼法併成一種）、第一組 hold-out 與私有系統的分數逐邊不變。過程中兩次差點退步（TeaStore 的 artifactId 撞上 compose 服務名、私有系統的「系統對外位址」被當成外部呼叫）都由回歸抓到並改掉。

致謝

〔AI 使用聲明：依 COMPSAC 規定，定稿時於此寫明使用的 AI 系統與受影響的章節。〕

參考文獻

[1] A. Al Maruf, A. Bakhtin, T. Cerny, and D. Taibi, “Using microservice telemetry data for system dynamic analysis,” in *Proc. IEEE SOSE*, 2022, pp. 29–38, doi: 10.1109/SOSE55356.2022.00010.

[2] Y. Tsubouchi, M. Furukawa, and R. Matsumoto, “Transtracer: Socket-based tracing of network dependencies among processes in distributed applications,” in *Proc. IEEE COMPSAC*, 2020, pp. 1206–1211, doi: 10.1109/COMPSAC48688.2020.00-92.

[3] V. Bushong, D. Das, and T. Cerny, “Reconstructing the holistic architecture of microservice systems using static analysis,” in *Proc. CLOSER*, 2022, pp. 149–157, doi: 10.5220/0011032100003200.

[4] S. Schneider and R. Scandariato, “Automatic extraction of security-rich dataflow diagrams for microservice applications written in Java,” *J. Syst. Softw.*, vol. 202, art. no. 111722, 2023, doi: 10.1016/j.jss.2023.111722.

[5] M. I. Rahman, S. Panichella, and D. Taibi, “A curated dataset of microservices-based systems,” in *Proc. SSSME*, CEUR-WS vol. 2520, 2019, pp. 1–9. [Online]. Available: https://arxiv.org/abs/1909.03249

[6] V. Liljas, M. Esposito, V. Lenarduzzi, and D. Taibi, “Can agents reconstruct microservice architecture?,” in *Proc. ESEM*, LIPIcs vol. 394, 2026, pp. 62:1–62:13, doi: 10.4230/LIPIcs.ESEM.2026.62.

[7] L. Sui, J. Dietrich, A. Tahir, and G. Fourtounis, “On the recall of static call graph construction in practice,” in *Proc. ICSE*, 2020, pp. 1049–1060, doi: 10.1145/3377811.3380441.

[8] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, and D. Taibi, “Comparison of static analysis architecture

recovery tools for microservice applications," *Empir. Softw. Eng.*, vol. 30, no. 5, art. 128, 2025, doi: 10.1007/s10664-025-10686-2.

[9] M. E. Gortney, P. E. Harris, T. Cerny, A. Al Maruf, M. Bures, D. Taibi, and P. Tisnovsky, "Visualizing microservice architecture in the dynamic perspective: A systematic mapping study," *IEEE Access*, vol. 10, pp. 119999–120012, 2022, doi: 10.1109/ACCESS.2022.3221130.

[10] B. Mayer and R. Weinreich, "An approach to extract the architecture of microservice-based software systems," in *Proc. IEEE SOSE*, 2018, pp. 21–30, doi: 10.1109/SOSE.2018.00012.

[11] G. Granchelli, M. Cardarelli, P. Di Francesco, I. Malavolta, L. Iovino, and A. Di Salle, "Towards recovering the software architecture of microservice-based systems," in *Proc. ICSAW*, 2017, pp. 46–53, doi: 10.1109/ICSAW.2017.48.

[12] J. Soldani, J. Khalili, and A. Brogi, "Offline mining of microservice-based architectures," in *Proc. CLOSER*, 2022, pp. 63–73, doi: 10.5220/0011061000003200.

[13] A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, and D. Taibi, "Tools reconstructing microservice architecture: A systematic mapping study," in *Software Architecture. ECSA 2023 Tracks, Workshops, and Doctoral Symposium*, LNCS 14590, 2024, pp. 3–18, doi: 10.1007/978-3-031-66326-0_1.

[14] O. Grinenko, A. Gizun, A. Morhun, and A. Bekh, "Methods for reconstructing microservice architectures: A thematic literature review," in *Proc. 5th Int. Workshop on Cyber Hygiene (WCH 2026)*, CEUR-WS vol. 4278, 2026, pp. 131–146. [Online]. Available: https://ceur-ws.org/Vol-4278/paper10.pdf

[15] Y. Zhang, Z. Xu, C. Liu, H. Chen, J. Sun, D. Qiu, and Y. Liu, "Software architecture recovery with information fusion," in *Proc. ESEC/FSE*, 2023, pp. 1535–1547, doi: 10.1145/3611643.3616285.

[16] S.-P. Ma, C.-Y. Fan, Y. Chuang, W.-T. Lee, S.-J. Lee, and N.-L. Hsueh, "Using service dependency graph to analyze and test microservices," in *Proc. IEEE COMPSAC*, vol. 02, 2018, pp. 81–86, doi: 10.1109/COMPSAC.2018.10207.

[17] S.-P. Ma, C.-Y. Fan, Y. Chuang, I.-H. Liu, and C.-W. Lan, "Graph-based and scenario-driven microservice analysis, retrieval, and testing," *Future Gener. Comput. Syst.*, vol. 100, pp. 724–735, 2019, doi: 10.1016/j.future.2019.05.048.

[18] S.-P. Ma, I.-H. Liu, C.-Y. Chen, J.-T. Lin, and N.-L. Hsueh, "Version-based microservice analysis, monitoring, and visualization," in *Proc. APSEC*, 2019, pp. 165–172, doi: 10.1109/APSEC48747.2019.00031.

[19] W.-K. Lin, S.-P. Ma, S.-J. Lee, and W.-T. Lee, "MicroViSim: Simulation and visualization of Kubernetes-based microservice systems," in *Proc. ASE Workshops (ASEW)*, 2025, pp. 161–164, doi: 10.1109/ASEW67777.2025.00038.

[20] S. Quadri, K. Kontogiannis, and D. Truong, "Architecture recovery revisited: A hybrid knowledge graph and LLM-based approach," in *Proc. IEEE COMPSAC*, 2026, pp. 493–502, doi: 10.1109/COMPSAC69091.2026.00070.

[21] M. Sterbentz, C. Barrie, S. Shahi, A. Dutta, D. Hooshmand, H. Pack, and K. J. Hammond, "Satyrn: A platform for analytics augmented generation," in *Proc. EMNLP*, 2024, pp. 6360–6385, doi: 10.18653/v1/2024.emnlp-main.365.

[22] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "PAL: Program-aided language models," in *Proc. ICML*, PMLR 202, 2023, pp. 10764–10799.

[23] G. C. Murphy, D. Notkin, and K. Sullivan, "Software reflexion models: Bridging the gap between source and high-level models," in *Proc. ACM SIGSOFT FSE*, 1995, pp. 18–28, doi: 10.1145/222124.222136.

[24] G. C. Murphy, D. Notkin, and K. J. Sullivan, "Software reflexion models: Bridging the gap between design and implementation," *IEEE Trans. Softw. Eng.*, vol. 27, no. 4, pp. 364–380, 2001, doi: 10.1109/32.917525.

[25] Datadog, "Service remapping rules," Datadog Documentation. Accessed: Oct. 13, 2026. [Online]. Available: https://docs.datadoghq.com/tracing/services/service_remapping_rules/

[26] C.-T. Lin, S.-P. Ma, and Y.-W. Huang, "MSABot: A chatbot framework for assisting in the development and operation of microservice-based systems," in *Proc. ICSE Workshops*, 2020, pp. 36–40, doi: 10.1145/3387940.3391501.

[27] Y.-W. Huang, S.-P. Ma, and S.-K. Wang, "MsdoBot: An extensible chabot [sic] platform for microservice development and operations," in *Proc. ICEBE*, 2022, pp. 124–129, doi: 10.1109/ICEBE55470.2022.00030.

[28] S.-K. Wang, S.-P. Ma, C.-H. Chao, and G.-H. Lai, "Low-code ChatOps for microservices systems using service composition," in *Proc. ICEBE*, 2023, pp. 55–62, doi: 10.1109/ICEBE59045.2023.00026.

[29] S.-K. Wang, S.-P. Ma, G.-H. Lai, and C.-H. Chao, "ChatOps for microservice systems: A low-code approach using service composition and large language models," *Future Gener. Comput. Syst.*, vol. 161, pp. 518–530, 2024, doi: 10.1016/j.future.2024.07.029.

[30] G.-H. Lai, S.-P. Ma, and A.-Y. Li, "LLM-powered ChatOps approach to microservice deployment pipeline automation and monitoring," in *Proc. IEEE Int. Conf. E-Business Engineering (ICEBE)*, 2025, pp. 24–31, doi: 10.1109/ICEBE68123.2025.00012.

[31] H. Guo, J. Huang, and D. H. Laidlaw, "Representing uncertainty in graph edges: An evaluation of paired visual variables," *IEEE Trans. Vis. Comput. Graph.*, vol. 21, no. 10, pp. 1173–1186, 2015, doi: 10.1109/TVCG.2015.2424872.

[32] M. Famelis, R. Salay, and M. Chechik, "Partial models: Towards modeling and reasoning with uncertainty," in *Proc. ICSE*, 2012, pp. 573–583, doi: 10.1109/ICSE.2012.6227159.

[33] Amazon Web Services, "AWS X-Ray concepts," AWS X-Ray Developer Guide. Accessed: Oct. 13, 2026. [Online]. Available: https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html